

Design and Implementation of a 32-Bit Arithmetic Logic Unit

EECE 7353 – VLSI Design, Northeastern University

Dylan Donahue
Northeastern University
Boston, MA
donahue.dyl@northeastern.edu

Billy Ferros
Northeastern University
Boston, MA
ferros.w@northeastern.edu

Hamza Iqbal
Northeastern University
Boston, MA
iqbal.ha@northeastern.edu

Abstract—This paper presents the design and implementation of a 32-bit Arithmetic Logic Unit (ALU) in the GPD045 45nm CMOS process using Cadence Virtuoso. The ALU supports arithmetic operations (addition, subtraction), bitwise logic operations (AND, OR, XOR, NOT), and barrel shift operations (logical left/right, arithmetic right). The adder is implemented using a Kogge-Stone parallel prefix topology and the shifter uses a 5-stage barrel architecture. Transistor sizing was performed using logical effort analysis implemented in MATLAB. Schematic designs were completed for all functional blocks. While we were unable to finish all simulations and layout design, we were able to simulate logic operations for all individual blocks and begin work on some layout.

I. INTRODUCTION

The arithmetic logic unit is a fundamental building block of any digital processor, responsible for performing arithmetic and logical operations on binary data.

With our design, we aimed to primarily optimize for speed, and spent significant effort on our sizing analysis in order to do so.

Throughout this report, we discuss our design rationale, our implementation, and the results we were able to gather.

II. ALU ARCHITECTURE

A. Supported Operations

The ALU supports the operations listed in Table I. A 4-bit opcode selects the operation, decoded by a control unit into internal control signals.

TABLE I: ALU Opcode Encoding

Opcode	Operation	Description
0000	ADD	$A + B$
0001	SUB	$A - B$ (two's complement)
0010	AND	$A \& B$
0011	OR	$A \mid B$
0100	XOR	$A \wedge B$
0101	NOT	$\sim A$
0110	SHL	$A \ll \text{shamt}$
0111	SHR	$A \gg \text{shamt}$ (logical)
1000	SHRA	$A \gg \text{shamt}$ (arithmetic)

B. Top-Level Block Diagram

The ALU is decomposed into the following functional blocks, shown in Fig. 1:

- **Control Unit:** Interprets flags and generates control signals
- **Kogge-Stone Adder:** 32-bit parallel prefix adder
- **Logic Unit:** Bitwise AND, OR, XOR, NOT
- **Barrel Shifter:** 5-stage NAND multiplexor shifter
- **Output Mux:** 3:1 result selection

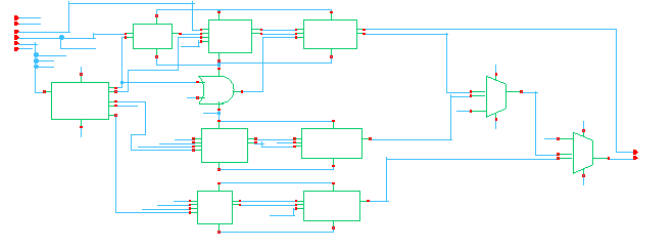


Fig. 1: Top-level ALU block diagram showing functional units and datapath.

C. Interface

- **Inputs:** $A[31:0]$, $B[31:0]$, $S[2:0]$, C_{in}
- **Outputs:** $Result[31:0]$, C_{out}

III. ADDER DESIGN

A. Design Choice Rationale

A Kogge-Stone adder tree was selected as the adder architecture. The Kogge-Stone adder tree optimizes the design for speed at the cost of area, and since the design rationale of this project was to optimize for speed the Kogge-Stone adder was seen as the obvious choice.

TABLE II: Adder Topology Comparison

Topology	Depth	Area	Fanout
Ripple Carry	$O(n)$	Low	Low
CLA	$O(\log n)$	Medium	Medium
Brent-Kung	$O(\log n)$	Medium	Low
Kogge-Stone	$O(\log n)$	High	High

B. Kogge–Stone Adder Architecture

The Kogge–Stone adder is a parallel-prefix adder that computes carry signals using a tree-based structure, enabling logarithmic delay with respect to the input word size. For a 32-bit implementation, the architecture is composed of three primary blocks: the generate/propagate (GP) block, the prefix network, and the sum generation block.

1) *Generate/Propagate Block*: The GP block computes the bitwise generate and propagate signals for each input bit i :

$$g_i = a_i b_i \quad (1)$$

$$p_i = a_i \oplus b_i \quad (2)$$

The generate signal g_i indicates that a carry will be produced at bit i regardless of the input carry, while the propagate signal p_i indicates that an incoming carry will pass through bit i . In this implementation, each GP cell consists of a NAND gate and an XNOR gate, with the NAND gate used to realize the generate function in negative logic and the XNOR gate used for the propagate function. Since the logic is implemented in CMOS using negative logic conventions, the outputs of the GP block are passed through an inverter stage to restore them to positive logic before being applied to the prefix network.

2) *Prefix Network*: The prefix network computes the carry signals using a recursive combination of generate and propagate pairs. For two adjacent groups of bits, the combined generate and propagate signals are given by:

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j} \quad (3)$$

$$P_{i:j} = P_{i:k} P_{k-1:j} \quad (4)$$

These equations determine whether a carry is generated within a group of bits or whether an incoming carry propagates through the entire group. The prefix structure evaluates these relationships in parallel, allowing carries to be computed across increasingly larger spans of bits at each stage.

The prefix network is composed of gray and black cells. Gray cells compute only the group generate signal $G_{i:j}$, while black cells compute both the group generate and propagate signals. In this design, gray cells are implemented using AOI logic, while black cells are implemented using a combination of an AOI gate and a NAND gate. The first prefix stage consists of two gray cells and thirty black cells, and subsequent stages follow the standard Kogge–Stone interconnection pattern. After each prefix stage, the outputs are inverted to restore the logic levels before being passed to the next stage.

The number of prefix stages is determined by the maximum distance over which carries must propagate. In a Kogge–Stone adder, each stage doubles the span of bits covered by the prefix operation. Therefore, for an N -bit adder, the number of stages required is $\log_2(N)$. For a 32-bit implementation, this results in five prefix stages:

$$\log_2(32) = 5 \quad (5)$$

After five stages, each bit position has access to all preceding generate and propagate information, allowing the carry signals to be fully resolved.

3) *Sum Generation Block*: Once all carry signals have been computed, the sum bits are generated using:

$$s_i = p_i \oplus c_i \quad (6)$$

In this implementation, the sum block consists of 32 XNOR gates followed by a final inverter stage to produce the correct output polarity. The propagate signals from the GP block and the carry signals from the prefix network are combined to generate the final sum outputs.

4) *Summary*: The Kogge–Stone adder achieves high performance by computing carry signals in a logarithmic number of stages using a parallel-prefix structure. The GP block encodes local carry behavior, the prefix network propagates this information across the entire word in $\mathcal{O}(\log N)$ time, and the sum block combines the resulting carry and propagate signals to produce the final output.

C. Subtraction

Subtraction is implemented via two's complement without a dedicated subtractor. A B-invert mux driven by `invert_b` computes $B' = \sim B$ when subtracting, and `cin_sel` forces $C_{in} = 1$, giving $A + \sim B + 1 = A - B$.

D. Schematic

The adder schematic hierarchy consists of: `gp_cell`, `black_cell`, `gray_cell`, and `sum_cell`.

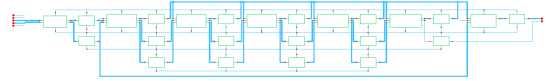


Fig. 2: Kogge–Stone 32-bit adder schematic.

IV. LOGIC UNIT DESIGN

A. Architecture

The logic unit performs bitwise operations on 32-bit operands. It is implemented as an array of 32 identical single-bit logic cells (`logic_cell`), each instantiated as `I<0:31>` in the parent schematic.

Each `logic_cell` contains:

- A 2-input AND gate (NAND2 + inverter)
- A 2-input OR gate (NOR2 + inverter)
- A 2-input XOR gate (static CMOS, complemented inputs)
- A NOT gate (inverter on A)
- A 4:1 transmission gate mux selecting the result
- One shared inverter generating A' for both NOT output and XOR input

The 4:1 mux is implemented as three 2:1 transmission gate muxes in a tree, with select inverters shared across level-1 stages.

B. Gate Implementation

Static CMOS was chosen for all logic gates for output drive strength and noise margin. Transmission gates were used for the output mux as they are appropriate for signal routing rather than logic computation. Table V summarizes baseline transistor sizing.

TABLE III: Logic Cell Baseline Transistor Sizing

Device	NMOS Width	PMOS Width
Logic gates (static CMOS)	180nm	360nm
TG mux stage 1	120nm	120nm
TG mux stage 2	120nm	120nm

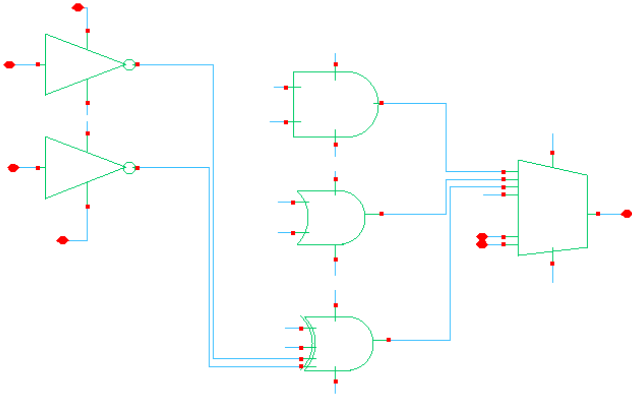


Fig. 3: `logic_cell` single-bit slice schematic showing all four operations and output mux.

V. BARREL SHIFTER DESIGN

A. Design Choice Rationale

A full NAND multiplexer chain was selected as the core architecture for the 32-bit barrel shifter because it provides an efficient balance of speed, robustness, and layout regularity in static CMOS design. A barrel shifter must perform variable shifts in a single clock cycle, making a staged multiplexer network the most practical implementation. By arranging the circuit into

$$\log_2(32) = 5$$

conditional shift stages corresponding to shifts of 1, 2, 4, 8, and 16 bits, any shift amount from 0 to 31 bits can be achieved with significantly less hardware than a full crossbar implementation. NAND gates were chosen to realize each 2:1 multiplexer because they are among the most efficient and fastest basic CMOS logic gates, offering low transistor count, strong drive capability, and full rail-to-rail voltage swing. Unlike pass-transistor implementations, the NAND-based mux does not suffer from threshold voltage degradation, resulting in improved noise margins and more reliable logic levels.

In addition, the repeated NAND mux cell structure creates a highly modular and scalable layout, which simplifies routing and transistor sizing across the 32-bit datapath. This regular structure also enables analytical optimization techniques such as logical effort to be applied for delay reduction and buffer sizing.

B. Architecture

The barrel shifter is implemented using seven arrays of thirty-two 2:1 multiplexers connected in sequence to perform both logical left and logical right shifts on a 32-bit input word. The first array functions as a bit-reversal stage, which is required for implementing left shifts using the same right-shift hardware path. In this stage, the least significant bit is swapped with the most significant bit, the second least significant bit is swapped with the second most significant bit, and this pattern continues across all thirty-two bits. Once reversed, the input can be processed through the shift network as a right shift. After the shifting operation is completed, the seventh and final array performs the reverse operation again, restoring the bit order and producing the correct left-shifted result. This method allows both left and right shifts to be implemented using a single shared shifting datapath, reducing circuit area and simplifying the overall design.

Between the two reversal stages are five multiplexing arrays that perform the variable shift operation. Each array corresponds to one control bit of the shift amount and conditionally shifts the data by a power of two. The first shifting stage performs a 16-bit shift, the second performs an 8-bit shift, followed by 4-bit, 2-bit, and finally 1-bit shifts. In each stage, every multiplexer receives two possible inputs: the A input is connected to the bit in the same position from the previous stage, while the B input is connected to the bit located a fixed distance to the right equal to that stage's shift amount. To implement a logical shift operation, positions that would require data from outside the 32-bit word are instead tied to ground. This forces a logic zero into any vacated bit positions as the data shifts, ensuring proper zero-fill behavior rather than wraparound or undefined values. For example, during the 16-bit shift stage, the leftmost sixteen multiplexers receive ground on their shifted-input path because no valid data exists beyond the word boundary. All thirty-two multiplexers within a stage share the same select signal, allowing that entire shift amount to be either enabled or bypassed simultaneously. The output of one stage feeds directly into the next stage, where the shift amount is reduced by half. By cascading these five conditional shift stages, any shift amount from 0 to 31 bits can be produced through the binary combination of the control signals. This staged multiplexer architecture provides a compact, fast, and scalable combinational implementation of a 32-bit barrel shifter.

C. Signal Degradation Consideration

Signal degradation was an important design consideration because data may propagate through several multiplexer stages in series before reaching the output. By implementing the

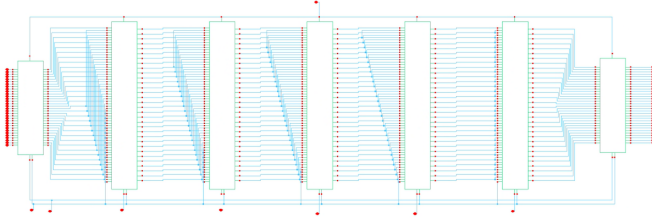


Fig. 4: 32-Bit Barrel Shifter

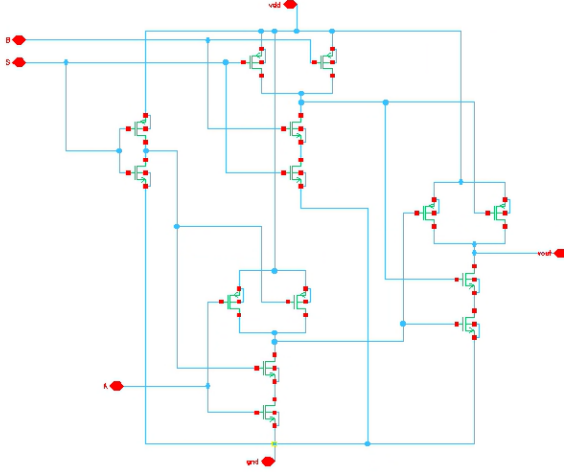


Fig. 5: 2:1 Mux Implemented by NAND

multiplexers with static CMOS NAND logic instead of pass-transistor logic, each stage restores the signal to full rail-to-rail voltage levels, preventing threshold-voltage loss and preserving noise margins. This is especially beneficial in a barrel shifter, where degraded logic levels could accumulate across multiple stages and cause slower transitions or incorrect outputs. Another concern is increased delay from capacitive loading and internal parasitics, particularly in later stages and on the high-fanout select lines. To reduce these effects, transistor sizing and buffered select paths were used to maintain faster edge transitions. In addition, unused shifted inputs were tied to ground so that vacated bit positions receive a defined logic zero rather than floating values. Together, these design choices improve signal integrity and ensure reliable operation of the barrel shifter.

VI. REMAINING BLOCKS

A. ALU Control Unit

The control unit decodes the 3-bit select input $s\langle 2 : 0 \rangle$ and generates the control signals required to enable the appropriate ALU sub-block. Its purpose is to ensure that only the desired functional unit is active for a given operation while also forwarding the relevant select information to the logic block when a logical operation is requested.

1) *Inputs*: The control unit takes a 3-bit control input:

- $s\langle 2 : 0 \rangle$: operation select input for the ALU.

This input is decoded combinatorially to generate one-hot-style enable signals for the arithmetic, shift, and logic datapaths.

2) *Outputs*: The control unit produces the following outputs:

- **logic_en**: enables the logic unit. When asserted, the ALU performs one of the logic functions selected by the lower bits of $s\langle 2 : 0 \rangle$.
- **shift_en**: enables the shift unit.
- **shift_left**: selects the shift direction. When asserted, the shift unit performs a left shift; otherwise it performs a right shift.
- **adder_en**: enables the adder unit.
- **sub**: selects subtraction within the arithmetic unit. When asserted together with **adder_en**, the datapath performs subtraction rather than addition.

For the logic operations, the control unit not only asserts **logic_en** but also forwards the lower two bits of the select input to the logic block. Thus, when $s\langle 2 \rangle = 0$, the logic unit is enabled and $s\langle 1 : 0 \rangle$ determines the specific logical function:

$$s\langle 1 : 0 \rangle = \begin{cases} 00 & \text{AND} \\ 01 & \text{OR} \\ 10 & \text{XOR} \\ 11 & \text{NOT} \end{cases} \quad (7)$$

3) *Control Decoder*: The overall mapping between the control input and the ALU function is summarized in Table IV.

TABLE IV: ALU control decode

$s\langle 2 : 0 \rangle$	logic_en	shift_en	shift_left	adder_en	sub	Operation
000	1	0	X	0	0	AND
001	1	0	X	0	0	OR
010	1	0	X	0	0	XOR
011	1	0	X	0	0	NOT
100	0	1	0	0	0	Shift right
101	0	1	1	0	0	Shift left
110	0	0	X	1	0	Add
111	0	0	X	1	1	Sub

4) *Summary*: The control unit decodes the ALU select input into a set of enable signals that activate the correct functional block. For logical operations, the control unit asserts **logic_en** and forwards $s\langle 1 : 0 \rangle$ to the logic unit to distinguish between AND, OR, XOR, and NOT. For shift operations, it asserts **shift_en** and uses **shift_left** to determine the shift direction. For arithmetic operations, it asserts **adder_en**, while the **sub** signal selects between addition and subtraction.

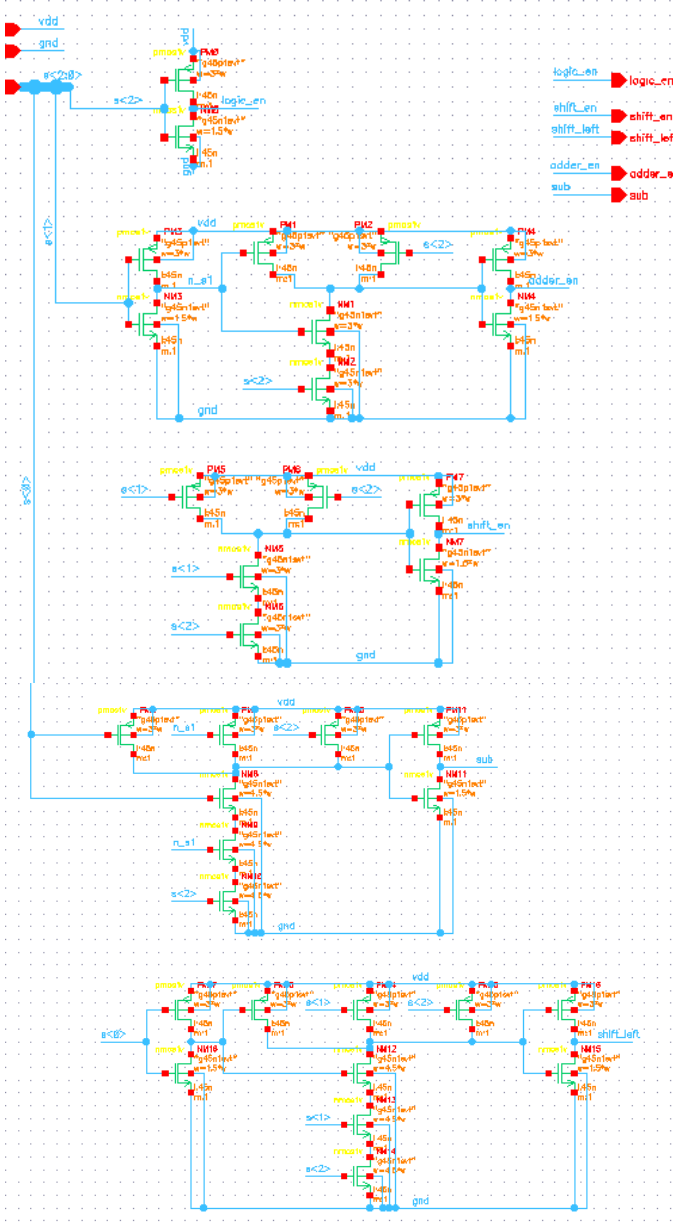


Fig. 6: control_blk Schematic view of control logic.

B. Output Mux

The final stage of the ALU consists of a two-level multiplexing network that selects the appropriate output from the arithmetic, logic, and shift units. This structure ensures that only one functional unit drives the final output at any given time, based on the control signals derived from $s\langle 2 : 0 \rangle$.

1) *Structure*: The output selection is implemented using two cascaded 2-to-1 multiplexers:

- The first multiplexer selects between the outputs of the **adder** and the **shift** unit.
- The second multiplexer selects between the output of the **logic** unit and the output of the first multiplexer.

This hierarchical arrangement reduces routing complexity and avoids the need for a larger multi-input multiplexer, which

would be more costly in terms of transistor count and delay.

2) *Operation*: The selection signals for the multiplexers are derived from the control input $s\langle 2 : 0 \rangle$:

- $s\langle 1 \rangle$ controls the first multiplexer, selecting between the adder and shift outputs.
- $s\langle 2 \rangle$ controls the second multiplexer, selecting between the logic output and the arithmetic/shift result.

This results in a two-stage selection process:

- 1) The intermediate result is formed by selecting either the adder output or the shift output.
- 2) The final output is selected between the logic output and the intermediate result.

3) *Functional Mapping*: The multiplexing structure aligns with the control unit operation as follows:

- When $s\langle 2 \rangle = 0$, the logic unit is selected, and the outputs of the arithmetic and shift units are ignored.
- When $s\langle 2 \rangle = 1$, the output is determined by the first multiplexer:
 - If $s\langle 1 \rangle = 0$, the shift unit output is selected.
 - If $s\langle 1 \rangle = 1$, the adder output is selected.

This selection scheme ensures compatibility with the overall ALU control encoding, where logical operations are grouped under $s\langle 2 \rangle = 0$, and arithmetic/shift operations are grouped under $s\langle 2 \rangle = 1$.

4) *Summary*: The use of two cascaded 2-to-1 multiplexers provides an efficient method for selecting among the outputs of multiple functional units. By structuring the selection hierarchically, the design minimizes hardware complexity while maintaining low delay. The control signals $s\langle 2 : 0 \rangle$ directly determine the data path through the multiplexers, ensuring correct routing of the selected ALU operation to the final output.

C. Functional Unit Enable Implementation

The ALU employs an input-gating strategy to control the activation of each functional unit. Rather than directly enabling or disabling entire blocks through clock gating or power gating, each unit is conditionally driven by its inputs using multiplexing logic. This approach ensures that only the selected functional unit receives valid operand data, while all other units receive inactive inputs.

1) *Structure*: Each functional block (logic, shift, and arithmetic) is preceded by a set of 2-to-1 multiplexers at its inputs. These multiplexers select between:

- The true input operands (e.g., $a\langle 31 : 0 \rangle$, $b\langle 31 : 0 \rangle$), and
- A constant zero value (ground).

The selection signal for each multiplexer is the corresponding enable signal generated by the control unit (e.g., logic_en, shift_en, adder_en).

2) *Operation*: The behavior of the input-gating scheme can be summarized as follows:

- When a functional unit is **enabled**, the multiplexers pass the true input operands into the block, allowing it to compute a valid output.

- When a functional unit is **disabled**, the multiplexers instead pass zeros (ground) to the inputs, forcing the internal logic to produce a neutral or inactive output.

For example:

- If `logic_en` is asserted, the logic unit receives the actual operand values and performs the selected logical operation.
- If `logic_en` is deasserted, the logic unit inputs are forced to zero, preventing it from contributing meaningful data to the output.

3) *Rationale*: This design technique provides several advantages:

- **Functional isolation**: Only the selected unit processes valid data, reducing the risk of unintended interactions between parallel datapaths.
- **Simplified control**: Enable signals directly control data flow without requiring additional gating of internal nodes.
- **Reduced switching activity**: Feeding zeros into inactive units minimizes unnecessary signal transitions, which can reduce dynamic power consumption.

4) *Summary*: The enable mechanism is implemented by multiplexing each functional unit's inputs with ground. When a unit is not selected, its inputs are forced to zero, effectively disabling its contribution to the ALU output. When selected, the true operand values are passed through, allowing normal operation. This approach provides a simple and effective method for controlling multiple parallel datapaths within the ALU.

VII. TRANSISTOR SIZING

A. Methodology

Transistor sizing was performed using the logical effort method [1]. As expected, this area is where we saw many of our challenges, and spent lots of our effort in trying to continuously improve our results here.

Notably, parts of our design combined transmission gate stages with traditional CMOS, and we had reconvergent fanout in a few locations.

For the adder and the logic unit, because of circular dependencies in our sizing calculation we determined that it was not feasible to calculate the sizing for each gate. To get around this restraint we had to use the optimization toolbox in Matlab to iteratively solve for ideal sizing and stage number. For the barrel-shifter, this was not necessary.

B. Logic Unit Sizing

When determining the sizing for the logic unit, the critical path through the unit was first identified by considering the entire signal chain from input to output. This critical path includes both the input enable multiplexers and the final output multiplexers, as these elements contribute to the overall propagation delay. The identified critical path is shown in Fig. 7.

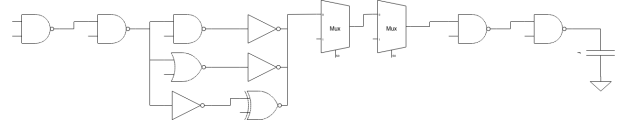


Fig. 7: Critical path through the logic unit including input enable multiplexers and output selection multiplexers.

To determine the optimal transistor sizing for the logic unit, a MATLAB-based optimization framework was developed to analyze the critical path identified in Fig. 7. The script evaluates the delay and sizing requirements across the entire signal chain, including the input enable multiplexers, the internal logic gates, and the final output multiplexers. The full implementation of this optimization procedure is provided in the appendix.

The sizing procedure is based on the principles of logical effort. Each gate along the critical path is characterized by its logical effort g , input capacitance C'_{in} , and the load it must drive. The script models the full critical path, which consists of a shared prefix, three branching logic paths, and a common tail section leading to the final load.

The optimization determines the optimal stage effort f such that the delay is balanced across all stages. Due to the presence of multiple branching paths, the script evaluates three candidate paths through the logic unit:

- Path A: NAND $\rightarrow$ INV $\rightarrow$ MUX
- Path B: NOR $\rightarrow$ INV $\rightarrow$ MUX
- Path C: INV $\rightarrow$ XOR $\rightarrow$ MUX

Each path includes the shared prefix and tail stages. The worst-case delay among these paths is used as the objective for optimization.

For a given stage effort f , the scale factor x_i for each stage is computed using:

$$x_i = \frac{g_i \cdot C_{out,i}}{f \cdot C_{in,i}} \quad (8)$$

The sizing is performed in a backward manner, beginning from the final load and propagating toward the input. To ensure physical feasibility, the following constraints are enforced:

- $x_i \geq 1$ (minimum device size),
- $x_i \leq x_{max}$ corresponding to a maximum transistor width of 10 μm .

The optimization produced balanced delays across all three paths:

$$\text{Path A} = 17.1838 \quad (9)$$

$$\text{Path B} = 17.1838 \quad (10)$$

$$\text{Path C} = 17.1838 \quad (11)$$

This confirms that the stage effort has been properly equalized, and no single path dominates the overall delay.

The resulting scale factors for each stage are summarized in Table V.

TABLE V: Optimal scale factors for logic unit critical path

Stage	Scale Factor x_i	Circuit Element
x_1	1.0000	Input MUX NAND1
x_2	2.3606	Input MUX NAND2
x_3	1.0000	Logic NAND
x_4	1.3312	Inverter (Path A)
x_5	1.0000	Logic NAND
x_6	1.3312	Inverter (Path B)
x_7	1.0000	Inverter (Path C)
x_8	1.3312	Logic XNOR
x_9	3.3835	Transmission Gate MUX 1
x_{10}	8.5997	Transmission Gate MUX 2
x_{11}	10.9289	Output MUX NAND1
x_{12}	13.8889	Output MUX NAND2

The results indicate that the earlier stages in the logic unit remain at minimum size, while later stages increase in size to effectively drive the larger downstream capacitance. This behavior is consistent with logical effort theory, where gate sizes increase progressively toward the output to minimize delay.

The equalization of delays across all three paths demonstrates that the optimization successfully accounts for branching effects within the logic unit. This is particularly important in this design, as multiple logic paths converge into a shared output multiplexing structure.

C. Adder Sizing

Due to the depth and complexity of the Kogge–Stone adder, determining optimal transistor sizing through manual analysis is impractical. The critical path spans multiple stages, including the generate/propagate block, five prefix stages, and the final sum logic, while also exhibiting significant branching due to the prefix tree structure. These characteristics introduce non-trivial interactions between stages, making it necessary to use an automated optimization framework.

To address this, a MATLAB-based script was developed to perform logical effort based optimization of the critical path. The implementation of this script is provided in the appendix.

The critical path used for optimization is shown in Fig. 8. This path includes the full chain from the initial generate/propagate computation through the prefix network and into the final sum stage.

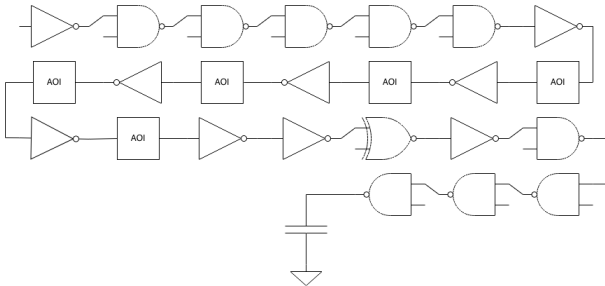


Fig. 8: Extracted critical path used for logical effort optimization.

The optimization framework models the critical path as a sequence of stages, each characterized by logical effort g_i ,

parasitic delay p_i , and input capacitance. In addition to the direct path, the script explicitly accounts for branching effects, where intermediate nodes drive both on-path and off-path loads. These effects are incorporated through the branching effort term:

$$B = \prod_i \frac{C_{\text{on},i} + C_{\text{off},i}}{C_{\text{on},i}} \quad (12)$$

A key parameter in the model is the shared load at the reconverging sum stage, denoted as C_{sum} . This parameter captures the impact of multiple branches converging at the final stage and is treated as an optimization variable.

The total path effort is defined as:

$$F = G \cdot B \cdot H \quad (13)$$

where G is the path logical effort, B is the branching effort, and H is the electrical effort.

For a path consisting of N stages, the optimal stage effort is given by:

$$f = F^{1/N} \quad (14)$$

The script sweeps over possible values of C_{sum} and evaluates the resulting delay. This relationship is illustrated in Fig. 9, which shows the delay as a function of C_{sum} . The optimal value corresponds to the minimum of this curve.

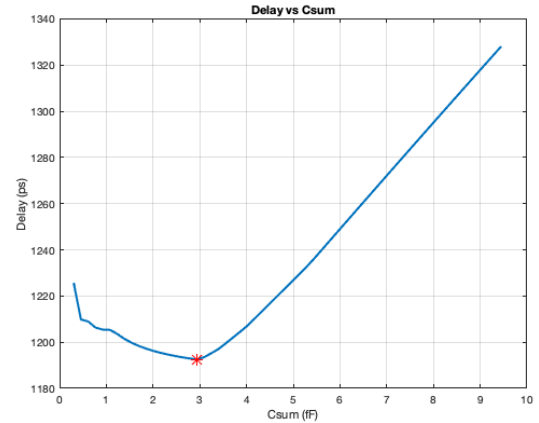


Fig. 9: Delay as a function of C_{sum} , showing the optimal operating point.

Once the optimal C_{sum} is determined, the script computes the scale factor for each stage using:

$$x_i = \frac{g_i \cdot C_{\text{out},i}}{f \cdot C_{\text{in},i}} \quad (15)$$

The sizing is performed in a backward manner starting from the output load and propagating toward the input. The resulting scale factors for all stages are shown in Fig. 10.

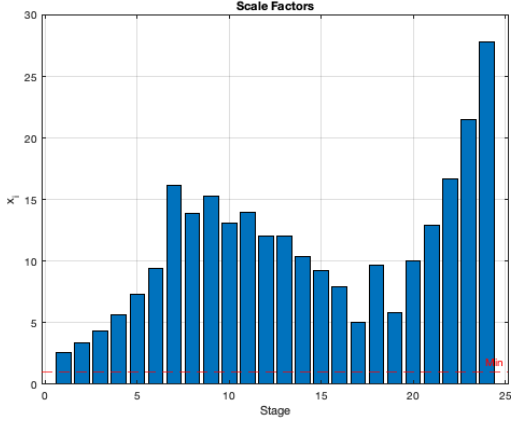


Fig. 10: Optimal scale factors for each stage along the critical path.

The total path effort is defined as:

$$F = G \cdot B \cdot H \quad (16)$$

where G is the path logical effort, B is the branching effort, and H is the electrical effort.

To ensure physical realizability, constraints are enforced such that:

- $x_i \geq 1$ (minimum device size),
- $x_i \leq x_{\max}$, corresponding to a maximum transistor width of 10 μm .

The optimization produced the following results:

$$C_{\text{sum}} = 2.9349 \text{ fF} \quad (17)$$

$$f = 1.7205 \quad (18)$$

$$D_{\min} = 1192.523 \text{ ps} \quad (19)$$

The corresponding optimal scale factors for each stage are summarized in Table VI.

TABLE VI: Optimal scale factors for adder critical path

Stage	Gate	Scale Factor x_i
1	inv0	2.6227
2	nand0	3.3844
3	nand1	4.3672
4	nand2	5.6354
5	nand3	7.2719
6	nand4	9.3837
7	inv1	16.1449
8	complex0	13.8889
9	inv2	15.2693
10	complex1	13.1357
11	inv3	13.9734
12	complex2	12.0209
13	inv4	12.0553
14	complex3	10.3708
15	inv5	9.2163
16	complex4	7.9284
17	inv6	5.0141
18	inv7	9.7054
19	xnor0	5.8236
20	inv8	10.0185
21	nand5	12.9278
22	nand6	16.6820
23	nand7	21.5265
24	nand8	27.7778

The results show a clear trend in which early stages remain relatively small while later stages grow significantly in size to drive the large output capacitance. This behavior is consistent with logical effort theory. Additionally, certain stages reach the maximum allowable size constraint, indicating that the delay is partially limited by physical constraints rather than purely logical optimization.

The convex shape of the delay curve in Fig. 9 confirms the existence of an optimal C_{sum} , demonstrating the importance of treating the reconverging load as an optimization variable.

D. Shifter Sizing

Because there were no reconvergent branches, the following calculations were done manually.

A 2:1 multiplexer is defined by

$$Y = A\bar{S} + BS$$

where S is the select signal, A is selected when $S = 0$, and B is selected when $S = 1$.

To implement this expression using only NAND gates, first rewrite it in a form that is compatible with DeMorgan's law. Starting with

$$Y = A\bar{S} + BS$$

apply double negation:

$$Y = \overline{\overline{A\bar{S} + BS}}$$

Using DeMorgan's law,

$$\overline{X + Y} = \bar{X}\bar{Y}$$

so

$$Y = \overline{\overline{A\bar{S}} \overline{BS}}$$

This is now in a form directly realizable with NAND gates. First, generate the complement of the select signal:

$$\bar{S} = \text{NAND}(S, S)$$

Next, form the two product terms using NAND gates:

$$X_1 = \overline{A\bar{S}} = \text{NAND}(A, \bar{S})$$

$$X_2 = \overline{BS} = \text{NAND}(B, S)$$

Finally, NAND these two intermediate signals together:

$$Y = \overline{X_1 X_2}$$

Substituting for X_1 and X_2 ,

$$Y = \overline{(\overline{A\bar{S}})(\overline{BS})}$$

which simplifies back to

$$Y = A\bar{S} + BS$$

Therefore, the mux can be implemented using only NAND gates as

$$Y = \text{NAND}(\text{NAND}(A, \bar{S}), \text{NAND}(B, S))$$

with

$$\bar{S} = \text{INV}(S)$$

This requires a total of 3 NAND gates and 1 INV gate:

- 1) one INV to generate $\bar{S}$
- 2) one NAND for $A\bar{S}$
- 3) one NAND for BS
- 4) one final NAND to combine the two terms

For the worst-case data path, the path effort is

$$F = GBH$$

where

$$G = \left(\frac{16}{9}\right)^{10}$$

$$B = 2^5 = 32$$

and

$$H = \frac{C_{out}}{C_{in}}$$

Using

$$C_{out} = 20 \text{ fF} \quad \text{and} \quad C_{in} = \frac{16}{9}(0.34 \text{ fF}) \approx 0.60 \text{ fF}$$

gives

$$H = \frac{20}{0.60} \approx 33$$

Therefore,

$$F = \left(\frac{16}{9}\right)^{10} (32)(33) \approx 3.32 \times 10^5$$

The optimal stage effort is

$$f = F^{1/10}$$

so

$$f = (3.32 \times 10^5)^{1/10} \approx 3.58$$

Using

$$h_i = \frac{f}{g_i b_i}$$

and taking the mux logical effort as

$$g_i = \frac{16}{9}$$

the sizing progression is approximated as follows:

$$\text{Stage 10} \approx \frac{33(16/9)}{3.58} \approx 16$$

$$\text{Stage 9} \approx \frac{16(16/9)}{3.58} \approx 8$$

$$\text{Stage 8} \approx \frac{8(16/9)}{3.58} \approx 4$$

For branching stages, where $b = 2$,

$$h_i = \frac{f}{(16/9)(2)}$$

which keeps the branching stages near the same size. Hence the middle branching stages are kept near $4\times$.

A practical mux sizing choice is therefore:

$$[1, 2, 4, 4, 4, 4, 4, 8, 8, 16]$$

from Stage 1 to Stage 10.

The select input of the NAND-based mux drives one inverter input and one NAND input, so the effective select input capacitance is

$$C_{in,select} = C_{inv} + C_{nand}$$

Using a unit inverter input capacitance of

$$C_{inv} = 0.34 \text{ fF}$$

and a NAND input capacitance of

$$C_{nand} = \frac{4}{3}(0.34 \text{ fF}) = 0.453 \text{ fF}$$

gives

$$C_{in,select} = 0.34 + 0.453 = 0.793 \text{ fF}$$

For a 32-mux array with mux size scaled by 4,

$$C_L = 32(0.793)(4) \approx 101 \text{ fF}$$

Thus the electrical effort for the select buffer chain is

$$H = \frac{101}{0.34} \approx 298$$

The ideal number of inverter stages is estimated from

$$N \approx \ln(H)$$

which gives

$$N \approx \ln(298) \approx 5.69$$

So a 6-stage inverter chain is a reasonable choice.

The stage taper is then

$$f = 298^{1/6} \approx 2.58$$

Thus the inverter chain sizes are approximately

1, 2.58, 6.66, 17.2, 44.36, 114.5

which may be rounded for layout convenience.

VIII. SIMULATION RESULTS

A. Simulation Setup

All simulations were performed using Cadence Spectre in the GPDK045 45nm process at nominal conditions (VDD = 1.2V, T = 27°C, TT corner).

B. Functional Verification

Each block was verified independently prior to integration. Table VII summarizes propagation delay for each gate for the most meaningful gates/blocks in the system, all at unit sized/prior to logical effort scaling.

While anecdotal results were gathered for all components, the only simulation results formalized relate to the logic unit.

TABLE VII: Block-Level Simulation Results

Block	t_{pd} (ps)
AND2	22
OR2	23
XOR2	22
Logic cell	178
Adder	not completed
Barrel shifter	not completed

C. Waveforms

Again, only formalized waveforms were generated for the logic unit.

Here, a transient simulation that gave progressively longer pulse widths to each of a, b, and the two select bits allows the output to be verified across every single input combination. While in this simulation, the output does not have enough time to reach true high or low, the direction of movement is enough to signify the logical output.

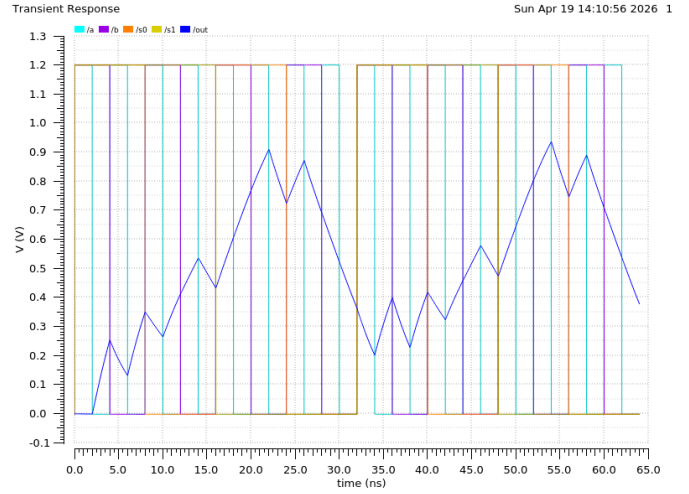


Fig. 11: Logic cell transient simulation showing all four operations across input transitions.

IX. WORK DIVISION CONCLUSION

We identified the adder, shifter, and logic units as the three "core" aspects of our design, and thus, each one of us took ownership of designing one of them. Hamza designed the Kogge-Stone adder, Billy designed the barrel-shifter, and Dylan designed the logic unit. For each of these, this included schematic design, implementing sizing changes, and any simulation work done.

The remaining work was generally done together as a group, but can be simplified a bit: Hamza led the effort on sizing calculations and writing the MATLAB scripts, Billy worked to gather top-level simulation results, and Dylan did most work on this report.

Our emphasis on sizing math lead us to have what we felt was a decently optimized design, though it did prevent us from gathering complete simulation results for all components, and did not allow us time to attempt layout. Ideally, we would continue to complete those efforts for our entire design.

REFERENCES

- [1] I. Sutherland, B. Sproull, and D. Harris, *Logical Effort: Designing Fast CMOS Circuits*. Morgan Kaufmann, 1999.
- [2] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2010.
- [3] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," *IEEE Trans. Comput.*, vol. C-22, no. 8, pp. 786–793, Aug. 1973.

APPENDIX

Additional transistor-level views of our designs are included below.

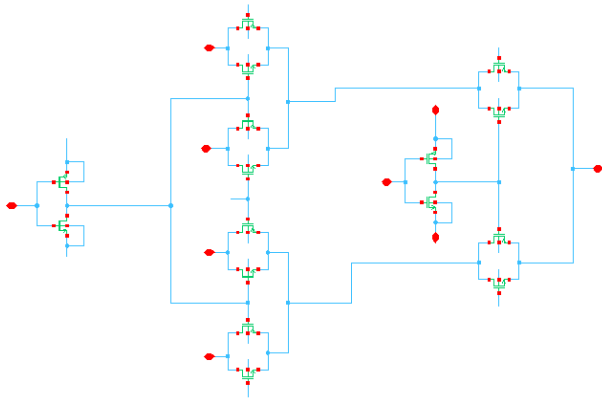


Fig. 12: 4:1 transmission gate mux.



Fig. 13: Top-level Kogge–Stone adder schematic.

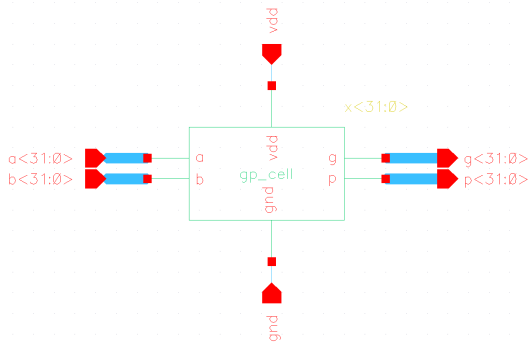


Fig. 14: Generate/Propagate (GP) block.

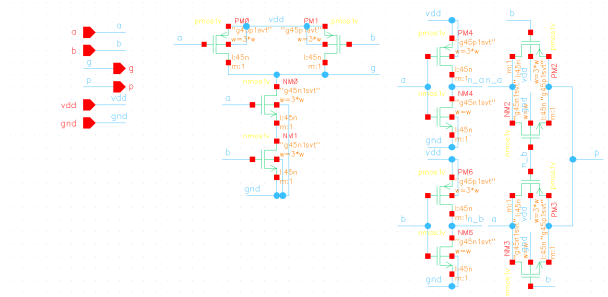


Fig. 15: GP cell implementation.

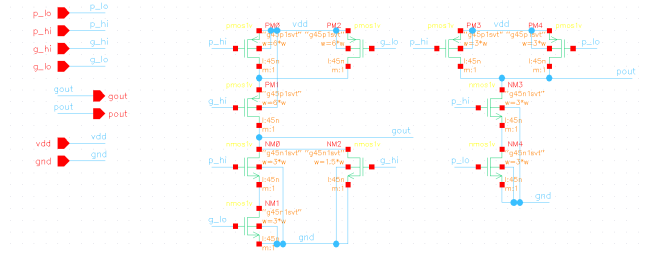


Fig. 16: Black cell used in the prefix network.

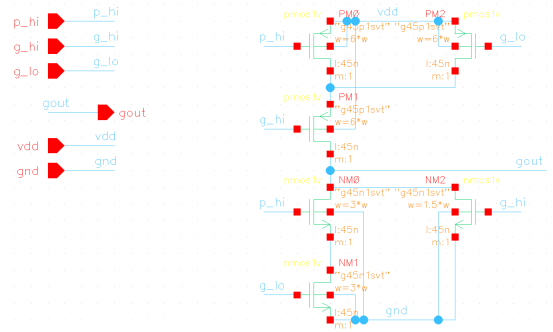


Fig. 17: Gray cell used in the prefix network.

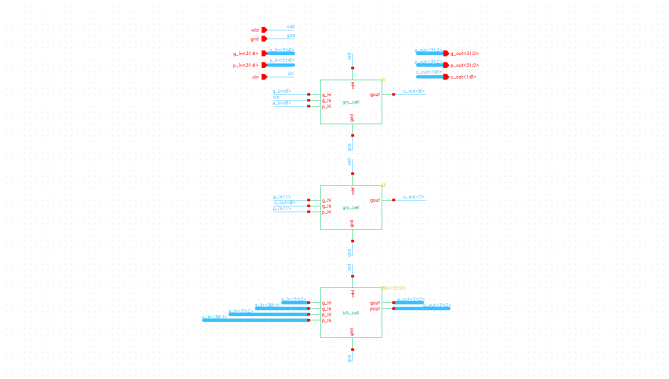


Fig. 18: Prefix stage 1.

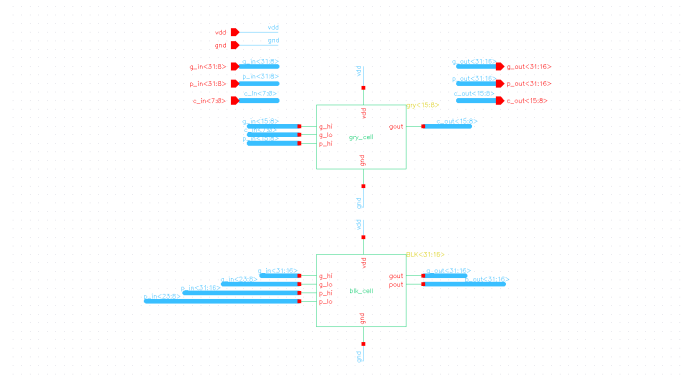


Fig. 21: Prefix stage 4.

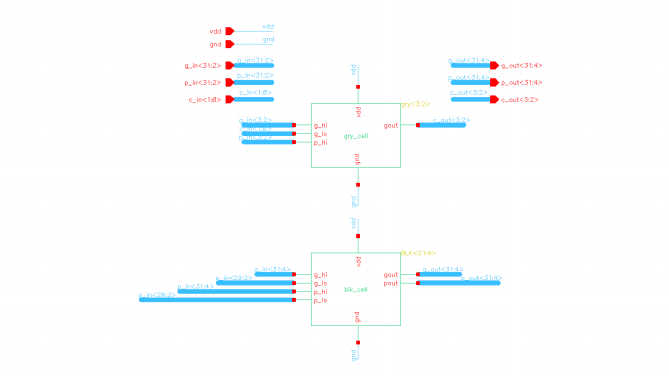


Fig. 19: Prefix stage 2.

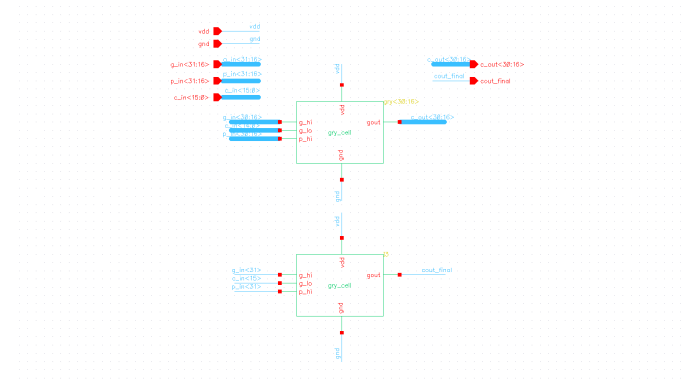


Fig. 22: Prefix stage 5.

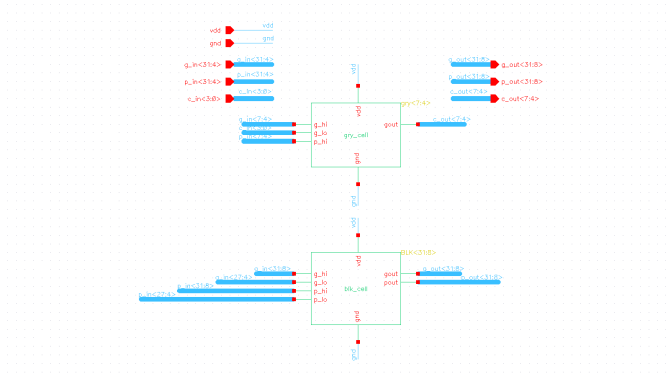


Fig. 20: Prefix stage 3.

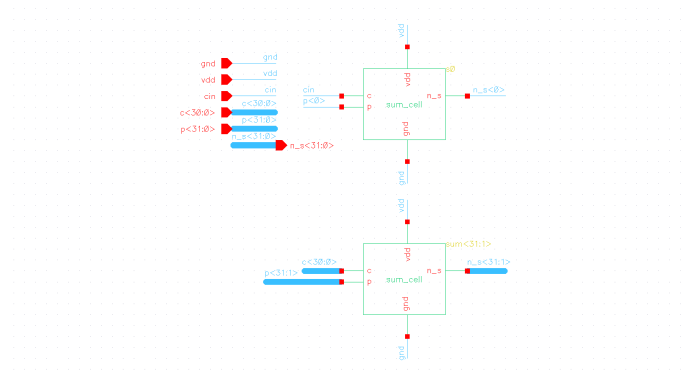


Fig. 23: Sum block.

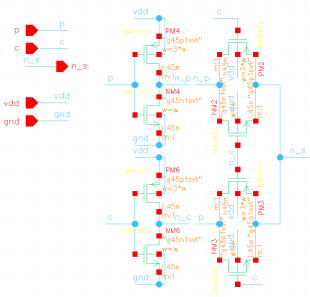


Fig. 24: Sum cell implementation.

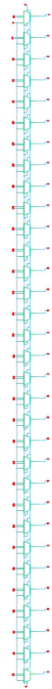


Fig. 25: 32-bit mux array for building the stage blocks of the barrel shifter. Selects are tied together for simultaneous shifting.

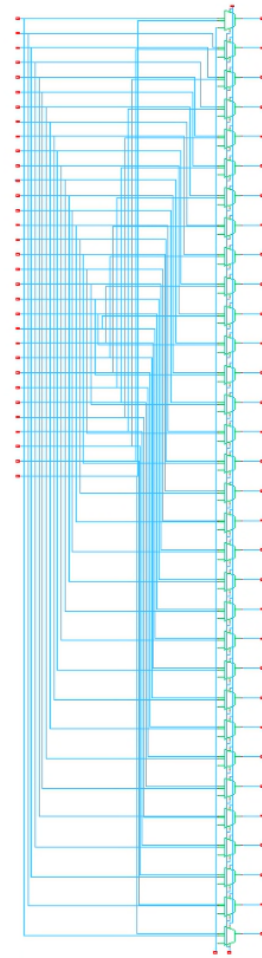


Fig. 26: 32-bit reversal block implemented with 2:1 multiplexers. The MSB connects to the A input of the first mux and B input of the last, repeating the pattern for full bit reversal.

The following MATLAB script implements the logical effort based sizing optimization for the logic unit critical path. The script determines the optimal stage effort, evaluates worst-case delay across multiple logic paths, and computes transistor scaling factors while enforcing physical design constraints.

```

1
2
3 %% =====
4 % CAPACITANCES (fF)
5 %% =====
6 Cnand = 0.4536;
7 Cnor = 0.567;
8 Cinv = 0.3402;
9 Cxor = 0.3402;
10 Cmux = 0.2268;
11
12 CL = 20; % 20 fF
13
14 gatecap.nand = Cnand;
15 gatecap.nor = Cnor;
16 gatecap.inv = Cinv;
17 gatecap.xor = Cxor;
18 gatecap.mux = Cmux;
19
20 %% =====

```

```

21 % LOGICAL EFFORTS
22 %% =====
23 g_nand = Cnand / Cinv;
24 g_nor = Cnor / Cinv;
25 g_inv = 1;
26 g_xor = Cxor / Cinv;
27 g_mux = Cmux / Cinv;
28
29 gle.nand = g_nand;
30 gle.nor = g_nor;
31 gle.inv = g_inv;
32 gle.xor = g_xor;
33 gle.mux = g_mux;
34
35 %% =====
36 % SIZE CONSTRAINTS
37 %% =====
38 WMAX_NM = 10000; % 10 um max transistor width
39 WUNIT_NM = 120; % base width quantum
40 CG_UNIT_FF = 0.0756; % gate cap for 120nm x 45nm
41
42 use_exact_width_caps = false;
43
44 if use_exact_width_caps
45     Wref.inv = 360;
46     Wref.nand = 720;
47     Wref.nor = 960;
48     Wref.xor = 720;
49     Wref.mux = 360;
50 else
51     Wref.inv = (Cinv / CG_UNIT_FF) * WUNIT_NM;
52     Wref.nand = (Cnand / CG_UNIT_FF) * WUNIT_NM;
53     Wref.nor = (Cnor / CG_UNIT_FF) * WUNIT_NM;
54     Wref.xor = (Cxor / CG_UNIT_FF) * WUNIT_NM;
55     Wref.mux = (Cmux / CG_UNIT_FF) * WUNIT_NM;
56 end
57
58 xcap.inv = WMAX_NM / Wref.inv;
59 xcap.nand = WMAX_NM / Wref.nand;
60 xcap.nor = WMAX_NM / Wref.nor;
61 xcap.xor = WMAX_NM / Wref.xor;
62 xcap.mux = WMAX_NM / Wref.mux;
63
64 xmin = 1;
65
66 %% =====
67 % SWEEP N
68 %% =====
69 N_vals = 8:2:20;
70
71 D_vals = zeros(size(N_vals));
72 f_vals = zeros(size(N_vals));
73 all_designs = cell(size(N_vals));
74
75 for k = 1:length(N_vals)
76     N = N_vals(k);
77
78     obj = @(f) objective_delay(f, N, gatecap, gle,
79         xcap, xmin, CL);
80     [f_opt, d_opt] = fminbnd(obj, 1.01, 20);
81
82     f_vals(k) = f_opt;
83     D_vals(k) = d_opt;
84     all_designs{k} = build_design(f_opt, N, gatecap,
85         gle, xcap, xmin, CL);
86 end
87
88 [D_min, idx_min] = min(D_vals);
89 best_N_delay = N_vals(idx_min);
90
91 fprintf('\n=====DELAY_SWEEP=====\\n');
92 for i = 1:length(N_vals)
93     fprintf('N=%d,f=%.4f,Delay=%.4f\\n',
94         N_vals(i), f_vals(i), D_vals(i));

```

```

92 end
93
94 fprintf('\\nMinimum-delay_N=%d\\n', best_N_delay);
95 fprintf('Minimum_Delay=%.4f\\n', D_min);
96
97 valid_idx = D_vals <= 1.10 * D_min;
98 N_valid = N_vals(valid_idx);
99 designs_valid = all_designs(valid_idx);
100
101 results = struct([]);
102 for k = 1:length(N_valid)
103     des = designs_valid{k};
104     results(k).N = N_valid(k);
105     results(k).f = des.f;
106     results(k).delay = des.worstDelay;
107     results(k).area = des.area;
108     results(k).score = des.area / des.worstDelay;
109     results(k).x = des.x_all;
110     results(k).paths = des.pathDelays;
111 end
112
113 scores = [results.score];
114 [~, idx_best] = min(scores);
115 best = results(idx_best);
116
117 fprintf('\\n=====FINAL_SELECTION=====\\n');
118 fprintf('Chosen_N=%d\\n', best.N);
119 fprintf('Chosen_f=%.4f\\n', best.f);
120 fprintf('Worst_Delay=%.4f\\n', best.delay);
121
122 fprintf('Path_delays:\\n');
123 fprintf('Path_A=%.4f\\n', best.paths(1));
124 fprintf('Path_B=%.4f\\n', best.paths(2));
125 fprintf('Path_C=%.4f\\n\\n', best.paths(3));
126
127 fprintf('xi_values:\\n');
128 for i = 1:length(best.x)
129     fprintf('x%-2d=%.4f\\n', i, best.x(i));
130 end
131
132 %% =====
133 % FUNCTIONS (MUST BE LAST)
134 %% =====
135
136 function d = objective_delay(f, N, gatecap, gle,
137     xcap, xmin, CL)
138     if f <= 0
139         d = 1e12;
140         return;
141     end
142
143     des = build_design(f, N, gatecap, gle, xcap,
144         xmin, CL);
145
146     if any(isnan(des.pathDelays)) || any(isinf(des.
147         pathDelays))
148         d = 1e12;
149         return;
150     end
151
152     d = des.worstDelay;
153 end
154
155 function des = build_design(f, N, gatecap, gle, xcap
156     , xmin, CL)
157     num_tail = N - 4;
158     num_inv = N - 8;
159
160     if num_tail < 4 || num_inv < 0
161         des = bad_design(f);
162         return;
163     end
164
165     tailTypes = cell(1, num_tail);

```



```

162 tailTypes{1} = 'mux';
163 tailTypes{2} = 'mux';
164 for i = 3:(2 + num_inv)
165     tailTypes{i} = 'inv';
166 end
167 tailTypes{end-1} = 'nand';
168 tailTypes{end} = 'nand';
169
170 x_tail = zeros(1, num_tail);
171
172 tlast = tailTypes{end};
173 x_tail(end) = clip_size((get_g(tlast, gle) * CL)
174 / (f * get_cap(tlast, gatecap)), xmin,
175 get_xcap(tlast, xcap));
176
177 for i = num_tail-1:-1:1
178     thisType = tailTypes{i};
179     nextType = tailTypes{i+1};
180     x_tail(i) = clip_size((get_g(thisType, gle)
181 * x_tail(i+1) * get_cap(nextType,
182 gatecap)) / ...
183 (f * get_cap(thisType,
184 gatecap)), xmin,
185 get_xcap(thisType,
186 xcap));
187
188 end
189
190 x9 = x_tail(1);
191
192 x4 = clip_size((gle.inv * x9 * gatecap.mux) / (
193 f * gatecap.inv), xmin, xcap.inv);
194
195 x3 = clip_size((gle.nand * x4 * gatecap.inv) / (
196 f * gatecap.nand), xmin, xcap.nand);
197
198 x6 = clip_size((gle.inv * x9 * gatecap.mux) / (
199 f * gatecap.inv), xmin, xcap.inv);
200
201 x5 = clip_size((gle.nor * x6 * gatecap.inv) / (
202 f * gatecap.nor), xmin, xcap.nor);
203
204 x8 = clip_size((gle.xor * x9 * gatecap.mux) / (
205 f * gatecap.xor), xmin, xcap.xor);
206
207 x7 = clip_size((gle.inv * x8 * gatecap.xor) / (
208 f * gatecap.inv), xmin, xcap.inv);
209
210 load2 = x3 * gatecap.nand + x5 * gatecap.nor +
211 x7 * gatecap.inv;
212
213 x2 = clip_size((gle.nand * load2) / (f * gatecap
214 .nand), xmin, xcap.nand);
215
216 x1 = 1;
217
218 d1 = gle.nand * (x2 * gatecap.nand) / (x1 *
219 gatecap.nand);
220
221 d2 = gle.nand * load2 / (x2 * gatecap.nand);
222
223 d3 = gle.nand * (x4 * gatecap.inv) / (x3 *
224 gatecap.nand);
225
226 d4 = gle.inv * (x9 * gatecap.mux) / (x4 *
227 gatecap.inv);
228
229 d5 = gle.nor * (x6 * gatecap.inv) / (x5 *
230 gatecap.nor);
231
232 d6 = gle.inv * (x9 * gatecap.mux) / (x6 *
233 gatecap.inv);
234
235 d7 = gle.inv * (x8 * gatecap.xor) / (x7 *
236 gatecap.inv);
237
238 d8 = gle.xor * (x9 * gatecap.mux) / (x8 *
239 gatecap.xor);
240
241 d_tail = zeros(1, num_tail);
242 for i = 1:num_tail
243     thisType = tailTypes{i};
244     if i == num_tail
245         load_i = CL;
246     else
247         nextType = tailTypes{i+1};
248         load_i = x_tail(i+1) * get_cap(nextType,
249 gatecap);
250     end
251     d_tail(i) = get_g(thisType, gle) * load_i /
252 (x_tail(i) * get_cap(thisType, gatecap));
253
254 end
255
256 d_tail_sum = sum(d_tail);
257
258 pathA = d1 + d2 + d3 + d4 + d_tail_sum;
259 pathB = d1 + d2 + d5 + d6 + d_tail_sum;
260 pathC = d1 + d2 + d7 + d8 + d_tail_sum;
261
262 area = 0;
263 area = area + x1 * gatecap.nand + x2 * gatecap.
264 nand;
265 area = area + x3 * gatecap.nand + x4 * gatecap.
266 inv;
267 area = area + x5 * gatecap.nor + x6 * gatecap.
268 inv;
269 area = area + x7 * gatecap.inv + x8 * gatecap.
270 xor;
271
272 for i = 1:num_tail
273     area = area + x_tail(i) * get_cap(tailTypes{
274 i}, gatecap);
275 end
276
277 des.f = f;
278 des.N = N;
279 des.area = area;
280 des.pathDelays = [pathA, pathB, pathC];
281 des.worstDelay = max(des.pathDelays);
282 des.x_all = [x1, x2, x3, x4, x5, x6, x7, x8,
283 x_tail];
284
285 end
286
287 function y = clip_size(x, xmin, xmax)
288     y = min(max(x, xmin), xmax);
289 end
290
291 function g = get_g(type, gle)
292     switch type
293     case 'nand', g = gle.nand;
294     case 'nor', g = gle.nor;
295     case 'inv', g = gle.inv;
296     case 'xor', g = gle.xor;
297     case 'mux', g = gle.mux;
298     end
299 end
300
301 function c = get_cap(type, gatecap)
302     switch type
303     case 'nand', c = gatecap.nand;
304     case 'nor', c = gatecap.nor;
305     case 'inv', c = gatecap.inv;
306     case 'xor', c = gatecap.xor;
307     case 'mux', c = gatecap.mux;
308     end
309 end
310
311 function x = get_xcap(type, xcap)
312     switch type
313     case 'nand', x = xcap.nand;
314     case 'nor', x = xcap.nor;
315     case 'inv', x = xcap.inv;
316     case 'xor', x = xcap.xor;
317     case 'mux', x = xcap.mux;
318     end
319 end

```

```

279 function des = bad_design(f)
280     des.f = f;
281     des.N = NaN;
282     des.area = 1e12;
283     des.pathDelays = [1e12, 1e12, 1e12];
284     des.worstDelay = 1e12;
285     des.x_all = [];
286 end
287 \end{verbatim}
288
289 \twocolumn
290
291
292 \clearpage
293 \onecolumn
294
295 \subsection{Adder Critical Path Sizing Optimization
296     Script}
297
298 The following MATLAB script implements logical
299 effort based optimization of the Kogge--Stone
300 adder critical path. The script determines the
301 optimal reconverging capacitance  $C_{\text{sum}}$ ,
302 computes the corresponding optimal stage
303 effort, and solves for transistor scale factors
304 while accounting for branching effects and
305 physical constraints.
306
307 \begin{verbatim}
308 clc; clear; close all;
309
310 %%
311
312 =====
313
314 % CONSTANTS
315 %%
316 =====
317
318 N = 24;
319 CL = 20; % fF
320 CG_UNIT = 0.0756;
321 WUNIT = 120;
322 WMAX = 10000;
323 tau = 15; % ps
324
325 g = [1,4/3,4/3,4/3,4/3,4/3,...
326     1,2,1,2,1,2,1,2,...
327     1,0.889,0.889,1,4/3,4/3,4/3,4/3];
328
329 p = [1,2,2,2,2,2,...
330     1,2,1,2,1,2,1,2,...
331     1,1,2,1,2,2,2,2];
332
333 coeff = [4.5,6,6,6,6,6,...
334     4.5,9,4.5,9,4.5,9,...
335     4.5,9,4.5,9,4.5,4,...
336     4,4.5,6,6,6,6];
337
338 cin = coeff * CG_UNIT;
339
340 widest = [3,3,3,3,3,3,...
341     3,6,3,6,3,6,...
342     3,6,3,6,3,2.6667,...
343     2.6667,3,3,3,3,3];
344
345 xmax = WMAX ./ (widest * WUNIT);
346 xmin = 1;
347
348 idx_complex = [8 10 12 14 16];
349 idx_next = [9 11 13 15 17];
350 idx_sum = 18;
351
352 gate_names = { ...
353     'inv0','nand0','nand1','nand2','nand3','nand4',
354     'inv1','complex0','inv2','complex1','inv3',
355     'complex2',...
356     'inv4','complex3','inv5','complex4','inv6','inv7',
357     '...',
358     'xnor0','inv8','nand5','nand6','nand7','nand8'};
359
360 %%
361 =====
362
363 % SWEEP Csum
364 %%
365 =====
366
367 Cmin = cin(idx_sum);
368 Cmax = cin(idx_sum)*xmax(idx_sum);
369
370 Csweep = linspace(Cmin, Cmax, 60);
371 Dsweep = zeros(size(Csweep));
372
373 for k = 1:length(Csweep)
374     Dsweep(k) = delay_vs_Csum(Csweep(k), ...
375         N, g, p, cin, CL, idx_complex, idx_next,
376         idx_sum, xmax, tau);
377 end
378
379 %%
380 =====
381
382 % OPTIMIZE Csum
383 %%
384 =====
385
386 [Csum_opt, Dmin] = fminbnd(@(C) delay_vs_Csum(C, ...
387     N, g, p, cin, CL, idx_complex, idx_next, idx_sum,
388     xmax, tau), ...
389     Cmin, Cmax);
390
391 [x_opt, f_opt, stageEff, branchF] = solve_given_Csum(
392     Csum_opt, ...
393     N, g, cin, CL, idx_complex, idx_next, idx_sum,
394     xmax);
395
396 %%
397 =====
398
399 % PRINT RESULTS
400 %%
401 =====
402
403 fprintf('\n=====FINAL_RESULT=====\n');
404 fprintf('Optimal_Csum=%.6f fF\n', Csum_opt);
405 fprintf('Optimal_f=%.6f\n', f_opt);
406 fprintf('Minimum_Delay=%.3f ps\n', Dmin);
407
408 fprintf('\n=====SCALE_FACTORS=====\n')
409 ;
410 for i = 1:N
411
412     xi = x_opt(i);
413     Cin_i = xi * cin(i);
414
415     if abs(xi - 1) < 1e-6
416         status = 'MIN(x=1)';
417     elseif abs(xi - xmax(i)) < 1e-6
418         status = 'MAX(10um)';
419     else
420         status = 'free';
421     end
422
423     fprintf('%-10s:xi=%.4fCin=%.4f fF [%s]\n', ...
424         gate_names{i}, xi, Cin_i, status);
425 end

```

```

394 fprintf('\n=====SUMMARY=====\\n');
395 num_min = sum(abs(x_opt - 1) < 1e-6);
396 num_max = sum(abs(x_opt - xmax) < 1e-6);
397
398
399 fprintf('#_stages_at_MIN_size_=%d\\n', num_min);
400 fprintf('#_stages_at_MAX_size_=%d\\n', num_max);
401
402 %%
403 =====
404 % FUNCTIONS
405 %%
406 =====
407
408 function D = delay_vs_Csum(Csum,N,g,p,cin,CL,
409     idx_complex,idx_next,idx_sum,xmax,tau)
410
411 [x, ~] = solve_given_Csum(Csum,N,g,cin,CL,
412     idx_complex,idx_next,idx_sum,xmax);
413
414 D = 0;
415
416 for i=1:N
417
418     Cin_i = x(i)*cin(i);
419
420     if i==N
421         Cout = CL;
422     elseif any(i==idx_complex)
423         k = find(idx_complex==i,1);
424         Cout = x(idx_next(k))*cin(idx_next(k))
425             Csum;
426     else
427         Cout = x(i+1)*cin(i+1);
428     end
429
430     D = D + g(i)*(Cout/Cin_i) + p(i);
431 end
432
433 D = D*tau;
434 end
435
436 function [x,f,stageEff,branchF] = solve_given_Csum(
437     Csum,N,g,cin,CL,idx_complex,idx_next,idx_sum,
438     xmax)
439
440 f = fsolve(@(f) residual(f,Csum,N,g,cin,CL,
441     idx_complex,idx_next,idx_sum,xmax), ...
442     3, optimoptions('fsolve','Display','off'));
443
444 x = backsolve(f,Csum,N,g,cin,CL,idx_complex,
445     idx_next,idx_sum,xmax);
446
447 stageEff = zeros(1,N);
448 branchF = ones(1,N);
449
450 for i=1:N
451
452     Cin_i = x(i)*cin(i);
453
454     if i==N
455         Cout = CL;
456         b=1;
457     elseif any(i==idx_complex)
458         k = find(idx_complex==i,1);
459         Cnext = x(idx_next(k))*cin(idx_next(k));
460         Cout = Cnext + Csum;
461         b = Cout/Cnext;
462     else
463         Cout = x(i+1)*cin(i+1);
464         b=1;
465     end

```

```

457
458     stageEff(i) = g(i)*(Cout/Cin_i);
459     branchF(i)=b;
460 end
461
462 function r = residual(f,Csum,N,g,cin,CL,idx_complex,
463     idx_next,idx_sum,xmax)
464
465 x = backsolve(f,Csum,N,g,cin,CL,idx_complex,
466     idx_next,idx_sum,xmax);
467
468 B=1;
469 for k=1:length(idx_next)
470     Cnext = x(idx_next(k))*cin(idx_next(k));
471     B = B*((Csum+Cnext)/Cnext);
472 end
473
474 H = 58.78895;
475 G = 336.74;
476
477 r = f - (H*G*B)^(1/N);
478 end
479
480 function x = backsolve(f,Csum,N,g,cin,CL,idx_complex,
481     idx_next,idx_sum,xmax)
482
483 x=zeros(1,N);
484
485 x(idx_sum)=clip(Csum/cin(idx_sum), xmax(idx_sum)
486     );
487
488 x(N)=clip((g(N)*CL)/(f*cin(N)), xmax(N));
489
490 for i=N-1:-1:idx_sum+1
491     x(i)=clip((g(i)*x(i+1)*cin(i+1))/(f*cin(i)),
492         xmax(i));
493 end
494
495 for i=idx_sum-1:-1:1
496
497     if any(i==idx_complex)
498         k=find(idx_complex==i,1);
499         load = x(idx_next(k))*cin(idx_next(k)) +
500             Csum;
501     else
502         load = x(i+1)*cin(i+1);
503     end
504
505     x(i)=clip((g(i)*load)/(f*cin(i)), xmax(i));
506 end
507
508 end
509
510 function y = clip(val,xmax)
511     y = max(1,min(val,xmax));
512 end

```